Report of RL Project 2: Quadruped Gait Control Using Reinforcement Learning

Ashwik Manoj Tatineni, Raj Priyadarshi, Sai Pranay Reddy

***Abstract*—This report documents our implementation of a reinforcement learning locomotion controller for the Unitree Go2 in Isaac Lab, following the provided tutorial progression (Parts 1–6). Starting from baseline velocity tracking, we progressively introduced action-rate regularization, an explicit PD torque controller, gait clock logic, Raibert heuristics, contact-aware shaping, and refined stabilization terms. Hyperparameters were tuned via parallel trial-and-error by team members and validated using TensorBoard curves (reward terms, terminations, stability indicators). As extra credit, actuator torque friction randomization was added to reduce jitter and improve robustness.**

I. Executive Summary

The objective of this project was to implement stable quadruped gait locomotion on plain terrain using reinforcement learning. The primary goal was to complete and extend the locomotion pipeline provided in the course repository by following tutorial steps (Parts 1–6).

As part of the extra credit, actuator torque friction randomization was implemented to reduce jitter and improve robustness. Attempts were also made to extend the framework to rough terrain and bipedal locomotion; however, these extensions were not fully successful and are not included as final results.

II. Project Requirements

The project requirements consisted of:

- Tasks defined in the provided GitHub repository
- Completion of tutorial Parts 1–6, including:
  - Action rate reward implementation
  - Reward logging updates
  - Reset logic updates
  - Explicit PD controller implementation
  - Control parameter initialization and torque limits
  - Early termination logic
  - Raibert heuristic implementation
  - Gait phase logic and foot indexing
  - Contact sensor integration
  - Reward refinement and tuning
- Fine tuning of reward parameters

III. Work Overview and Tuning Method

We implemented the tutorial sequentially from Part 1 to Part 6, and after each part we trained and evaluated policies.

Parameter selection method: values were first explored using trial-and-error. Runs were performed in parallel by team members (different random values and parameter settings). After obtaining a gait that was at least as good as the baseline, we relied on TensorBoard plots to judge whether a change was beneficial (e.g., stabilizing terms decreasing, fewer terminations, and smoother reward curves). This kind of parameter tuning was mostly performed on the parameters action rate reward scale , orient reward scale . Excpet these parameters most of the parameters were kept to their initial value as given in the project tutorial.

IV. Final Configurations

Two main configurations were used.

A. After Part 6 (Log file: 133345)

- lin_vel_reward_scale: 1.0 yaw_rate_reward_scale: 0.5
- action_rate_reward_scale: -0.15 orient_reward_scale: -5.0
- lin_vel_z_reward_scale: -0.1 dof_vel_reward_scale: -0.0001
- ang_vel_xy_reward_scale: -0.001
- raibert_heuristic_reward_scale: -10.0
- feet_clearance_reward_scale: -30.0
- tracking_contacts_shaped_force_reward_scale: 4.0
- Kp: 20.0 Kd: 0.5 torque_limits: 100.0

B. Extra Credit (Log file: 133736)

- action_rate_reward_scale: -0.05
- base_height_min: 0.2

V. Implementation Progression (Tutorial Parts)

A. PART 1: Baseline Setup and Velocity Tracking

Baseline training optimized only velocity tracking. This reproduced movement but produced jitter and weak gait structure.

Algorithm 1 PART 1: Baseline Reward Signals

Given commands $(v_x^{cmd}, v_y^{cmd}, \omega_z^{cmd})$
Measure base velocity (v_x, v_y, ω_z)
$r_{lin} \leftarrow \exp(-\|[v_x, v_y] - [v_x^{cmd}, v_y^{cmd}]\|^2/0.25)$
$r_{yaw} \leftarrow \exp(-(\omega_z - \omega_z^{cmd})^2/0.25)$
reward $\leftarrow$ lin_vel_reward_scale $\cdot r_{lin}$ + yaw_rate_reward_scale $\cdot r_{yaw}$

Design choice: keep rewards minimal first to reproduce baseline and identify instability sources.

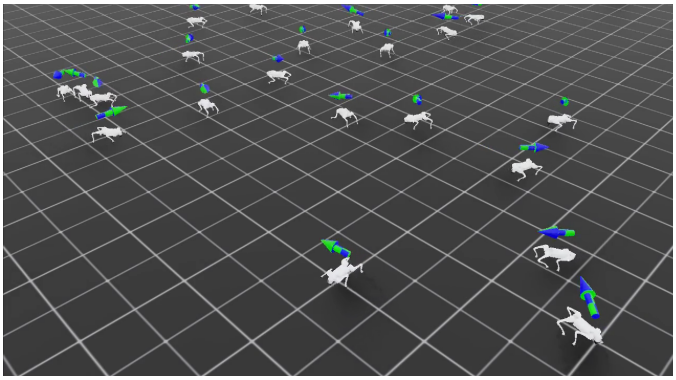

Fig. 1. Baseline Model. Robots are crawling and jittering

B. PART 2: Action-Rate Regularization + Logging

We added a penalty on rapid action changes using first and second differences. Logging was expanded to track each reward term separately. This reduced high-frequency jitter and made training signals easier to interpret.

Algorithm 2 PART 2: Action Rate Penalty (First + Second Difference)

1: Maintain action history buffer last_actions $\in \mathbb{R}^{N\times 12\times 3}$
2: $p_1 \leftarrow \|a_t - a_{t-1}\|^2$
3: $p_2 \leftarrow \|a_t - 2a_{t-1} + a_{t-2}\|^2$
4: rew_action_rate $\leftarrow (p_1 + p_2) \cdot (\text{action_scale}^2)$
5: reward $\leftarrow$ reward + action_rate_reward_scale $\cdot$ rew_action_rate
6: Log: Episode_Reward/rew_action_rate, Episode_Reward/track_lin_vel_xy_exp, Episode_Reward/track_ang_vel_z_exp, ...

Tuning note: action_rate_reward_scale was explored in parallel across team members (e.g., {-2, -1, -0.1, -0.01, -0.15}) and selected using TensorBoard trends (movement preserved, jitter reduced, fewer falls).

C. PART 3: Explicit PD Controller and Torque Limits

We disabled implicit gains and applied explicit PD torques. This prevents policies from exploiting hidden actuator dynamics.

Algorithm 3 PART 3: Explicit PD Torque Control

1: actions $a_t \in \mathbb{R}^{N\times 12}$ (joint position offsets)
2: $q_{des} \leftarrow q_{default} + \text{action_scale} \cdot a_t$
3: $\tau \leftarrow K_p \odot (q_{des} - q) - K_d \odot \dot{q}$
4: $\tau \leftarrow \texttt{clip}(\tau, -\text{torque_limits}, \text{torque_limits})$
5: apply τ to joints

Design choice: $K_p = 20.0$, $K_d = 0.5$ produced stable joint response; torque_limits bound unrealistic forces. These values are same as the default values.

D. PART 4: Gait Clock + Raibert Heuristic

We introduced gait phase variables and clock inputs, then added the Raibert heuristic reward to encourage a consistent stepping pattern and avoid crawling.

Algorithm 4 PART 4: Gait Phase and Clock Inputs

1: gait_indices $\in \mathbb{R}^N$; update $g \leftarrow (g + \Delta t \cdot f) \bmod 1$
2: Construct per-foot phases foot_indices $\in \mathbb{R}^{N\times 4}$
3: clock_inputs$[:, i] \leftarrow \sin(2\pi \cdot \text{foot_indices}[:, i])$
4: Append clock_inputs to observation (observation becomes 48+4)

Algorithm 5 PART 4: Raibert Foot Placement Reward

1: Compute current footsteps in body frame from foot_positions_w
2: Compute desired stance width/length nominal positions
3: Compute velocity-based offsets using commanded $(v_x^{cmd}, \omega_z^{cmd})$
4: error $\leftarrow$ desired_footsteps $-$ actual_footsteps
5: raibert_heuristic $\leftarrow \sum \|error\|^2$
6: reward $\leftarrow$ reward + raibert_heuristic_reward_scale $\cdot$ raibert_heuristic

Design choice: the heuristic provides structure early in training; weight was tuned to avoid "jumping" while still discouraging crawling.

E. PART 5: Contact Sensors + Foot Clearance + Shaped Contact Penalty

We integrated foot contact sensing to enforce swing/stance consistency. Foot clearance encouraged adequate swing height, while contact during swing was penalized using shaped force terms.

Algorithm 6 PART 5: Foot Clearance and Swing-Contact Shaping

1: Use desired_contact_states $\in \mathbb{R}^{N\times 4}$ from gait logic (0=swing, 1=stance)
2: Compute foot heights $h \in \mathbb{R}^{N\times 4}$
3: Define swing target $h^\star \leftarrow 0.08 \cdot \text{phase} + 0.02$
4: feet_clearance $\leftarrow \sum (h^\star - h)^2 \cdot (1 - \text{desired_contact_states})$
5: Measure foot forces $F \in \mathbb{R}^{N\times 4}$ from contact sensor
6: tracking_contacts_shaped_force $\leftarrow$ penalize F during swing
7: Add weighted terms to reward

Design choice: these terms remove foot scuffing and suppress "touching during swing", improving gait clarity. Furthermore the target height variable was changed to introduce randomness. However the training produced was not suitable for the rough terrain implementation. And therefore ignored.

F. PART 6: Reward Refinement + Early Termination

We added stabilization penalties (orientation, vertical velocity, joint velocity, roll/pitch angular velocity) and updated termination logic to avoid wasting rollouts on unrecoverable states.

Algorithm 7 PART 6: Full Reward Assembly and Early Termination

```
orient ← ‖projected_gravity_xy‖²
lin_vel_z ← (v_z)²
dof_vel ← Σ ‖q̇‖²
ang_vel_xy ← ‖[ω_x, ω_y]‖²
reward ← Σ_i w_i r_i over all terms (tracking, smoothness, gait, contacts)
if base contact force exceeds threshold then
    terminate
end if
if robot upside down OR base height < base_height_min then
    terminate
end if
```

Design choice: early termination increases sample efficiency and prevents learning unstable recovery behaviors. The default values of 1 N for contact forces were taken.

G. Extra Credit: Torque Friction Randomization

We randomized static and viscous friction coefficients per joint during reset. This reduced jitter and improved robustness.

Algorithm 8 Extra Credit: Actuator Friction Randomization at Reset

```
Sample fs_stiction ~ U(0.0, 0.3) per joint
Sample mu_viscous ~ U(0.0, 2.5) per joint
Compute friction torques: τ_s ← fs_stiction·tanh(q̇/0.1), τ_v ← mu_viscous · q̇
Optionally apply: τ ← τ − τ_s − τ_v
```

VI. Baseline Limitations

The baseline code produced high jerks and jittery motion due to missing action smoothing, missing contact-aware shaping, and limited reward structure. With only linear velocity and yaw rate tracking, optimization focused narrowly on command tracking and did not enforce physically consistent gait timing or smooth actuator behavior.

After progressively adding action-rate regularization, explicit PD control, gait heuristics, contact shaping, and stabilization penalties, the learned gait became noticeably smoother and more stable.

VII. Results and Validation Signals

We relied on TensorBoard to judge progress and stability:

- Episode reward terms (e.g., Episode_Reward/ang_vel_xy, orient, feet_clearance)
- Termination statistics (base_contact vs time_out)

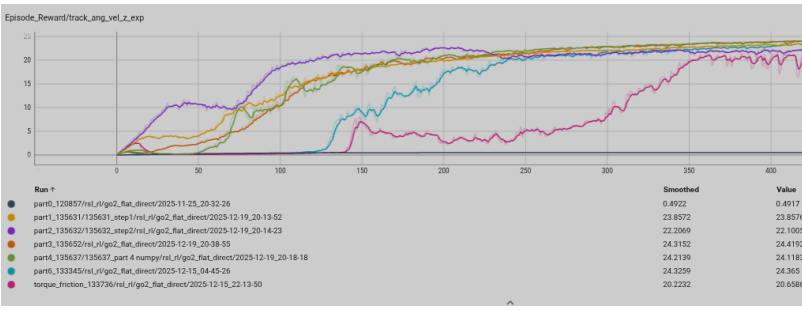

Fig. 2. Angular Velocity Z Exponential graphs. part 0 represent baseline model

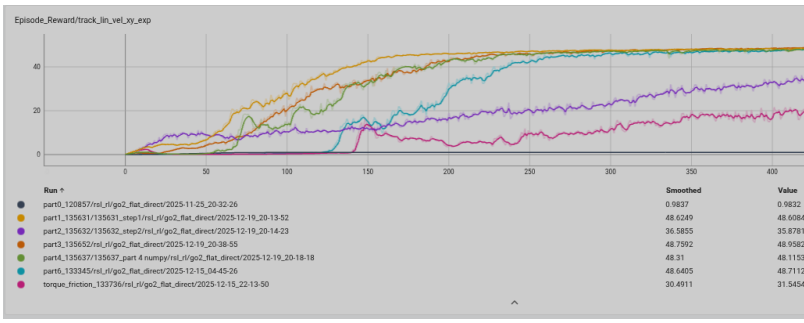

Fig. 3. Linear Velocity XY exponential graph

- Qualitative behavior from rollout videos (gait smoothness, diagonal pairing)

A configuration was considered "better" when it reduced instability/termination spikes while maintaining or improving command tracking. The following are the resulting graphs of various rewards.

VIII. Conclusion

We implemented tutorial Parts 1–6 to build a stable quadruped locomotion controller for Go2 on flat terrain. Parameters were tuned through parallel experimentation by team members and validated via TensorBoard analysis. The final policy achieved stable locomotion with reduced jitter.

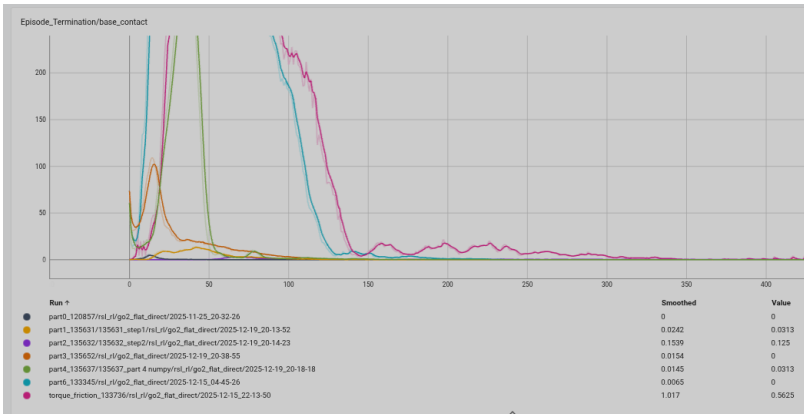

Fig. 4. Termination base contact

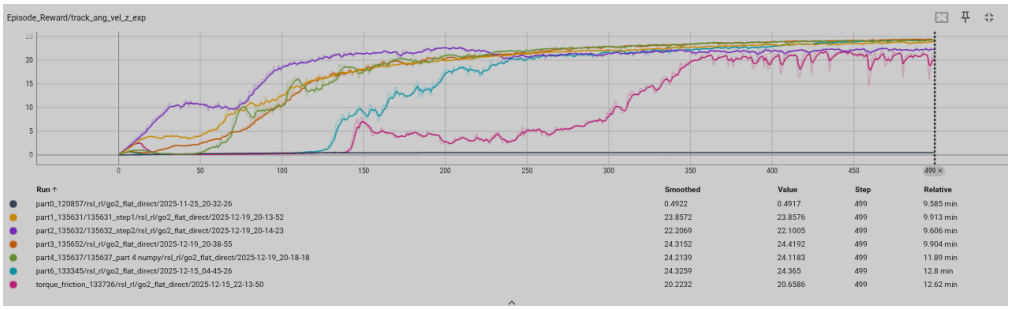

Fig. 5. Track ang vel z exp

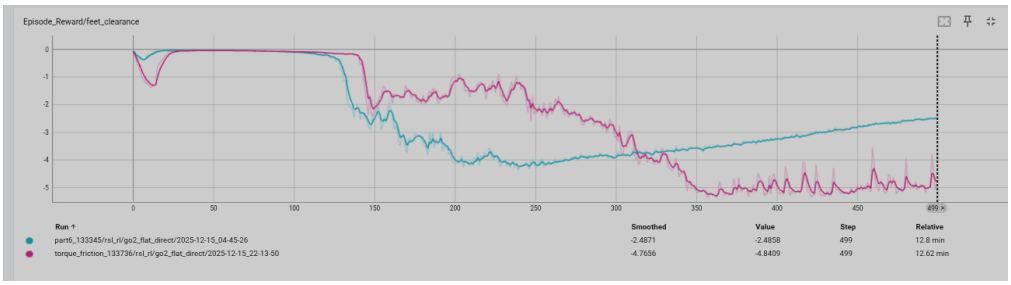

Fig. 6. Feet clearance

Torque friction randomization further improved robustness and reduced high-frequency oscillations. It is evident that training produced and model developed starts becoming more robust for the robot to move. The motion of the quadruped also becomes more and more smoother. However in step 4 , there is a noticeable drop in the motion quality of the system. This is due to the fact that a new constraints of the motion of the feets of the robot are introduced in form of gait structure. However no reward function was introduced at this step. Step 5 and Step 6 has been completed together in part 6. The extra credit part of friction randomization has also been implemented . Given by the torque friction 133736 plot.

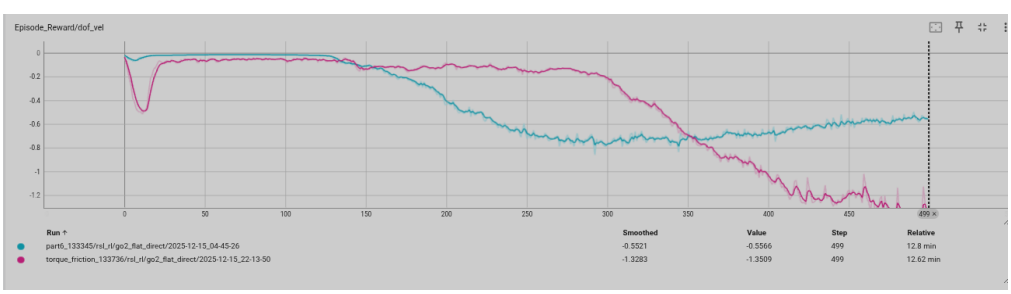

Fig. 7. Dof vel